

[Miejsce na: logo KIA (logo_kia.png)]

Wstaw logo wydziału/katedry

ZAAWANSOWANE PROGRAMOWANIE W JĘZYKU C++

RAPORT Z WYKONANIA PROJEKTU

GENCLIMB — APLIKACJA GTK4 DO GENEROWANIA TRAS WSPINACZKOWYCH KILTER

[Imię Nazwisko] – [numer indeksu] – [L00]

2026-06-09

Wstęp

Kilter Board to interaktywna ściana wspinaczkowa z podświetlanymi chwytami (dziurkami). Każda *trasa* to uporządkowany zbiór chwytów z przypisanymi rolami: start, chwyt pośredni, szczyt (finish) oraz chwyt na nogę. Niniejszy projekt laboratoryjny implementuje natywną aplikację C++ z interfejsem graficznym GTK4, która:

- wczytuje geometrię dziurek ściany z lokalnej bazy SQLite (`boardlib_data/kilter.sqlite3`),
- generuje nowe trasy przy użyciu wytrenowanego modelu (proces potomny Pythona),
- wizualizuje trasę na planszy,
- pozwala użytkownikowi ocenić trasę (1–5) i zapisać ją do lokalnej bazy wyników.

Kod aplikacji znajduje się w katalogu `kilter_cpp_gui/`. Projekt został zorganizowany modułowo: nagłówki w `include/`, implementacje w `src/`, konfiguracja budowania w `CMakeLists.txt`. Uruchomienie wymaga katalogu głównego repozytorium jako bieżącego katalogu roboczego (ścieżki względne do baz i skryptów Pythona).

Baza danych

Aplikacja korzysta z **dwóch instancji SQLite** — osobnej bazy tylko do odczytu oraz zapisywalnej bazy wyników użytkownika.

Baza planszy (`kilter.sqlite3`)

Baza *boardlib* zawiera co najmniej trzy tabele wykorzystywane przez zapytanie w module `board_db`:

- `holes` — identyfikator i współrzędne każdej dziurki na panelu,
- `placements` — powiązanie dziurki z układem (`layout_id`, `set_id`),
- `product_sizes_layouts_sets` — mapowanie wariantu produktu (`product_size_id`) na układ ściany.

Aplikacja odczytuje dziurki dla wybranego układu (token layoutu, np. 1270) poprzez złączenie (JOIN) tych tabel. Funkcja `product_size_id_from_layout_token` mapuje token modelu na identyfikator produktu w bazie `boardlib`.

Baza wyników (`saved_routes.db`)

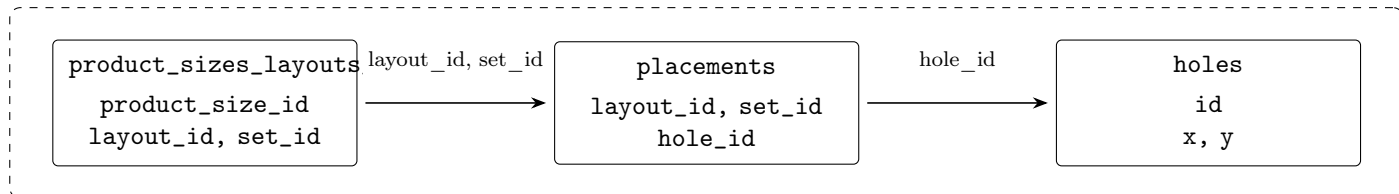
Zapisywalna baza tworzona automatycznie w `kilter_cpp_gui/data/` przechowuje historię tras zaakceptowanych przez użytkownika. Tabela `saved_routes` zawiera pola: `id`, `created_at`, `layout_id`, `grade_id`, `source` (model lub dataset), `rating`, `hole_count`, `tokens`, `hole_ids`.

Operacje na bazie wyników realizuje moduł `results_db`: inicjalizacja schematu (`init_results_db`), zapis (`save_route_to_db`) oraz odczyt ostatnich wpisów (`load_recent_routes`, sortowanie `ORDER BY id DESC`).

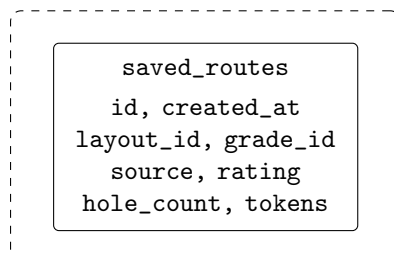
Diagram bazy danych

Na rysunku 1 przedstawiono uproszczony schemat relacji między tabelami obu baz. Strzałki oznaczają klucze obce logiczne wykorzystywane w zapytaniach aplikacji (niekoniecznie zadeklarowane jako FOREIGN KEY w pliku SQLite).

boardlib_data/kilter.sqlite3 (odczyt)



kilter_cpp_gui/data/saved_routes.db (zapis)



Rysunek 1: Uproszczony diagram tabel baz danych wykorzystywanych przez aplikację.

Implementacja — połączenie z bazą

Inicjalizacja lokalnej bazy wyników oraz odczyt dziurek planszy ilustrują listingi 1 i 2. Dostęp do SQLite realizowany jest przez bibliotekę libsqlite3 (CMake: PkgConfig::SQLITE3). Parametryzowane zapytania (sqlite3_prepare_v2, sqlite3_bind_*) zapobiegają wstrzyknięciu SQL.

```
1 bool init_results_db(const std::string& db_path, std::string& error) {
2     std::error_code ec;
3     fs::create_directories(fs::path(db_path).parent_path(), ec);
4
5     sqlite3* db = nullptr;
6     if (sqlite3_open(db_path.c_str(), &db) != SQLITE_OK) {
7         error = "Could not open results DB: " +
8             std::string(sqlite3_errmsg(db));
9         if (db) sqlite3_close(db);
10        return false;
11    }
12    const char* sql =
13        "CREATE TABLE IF NOT EXISTS saved_routes ("
14        " id INTEGER PRIMARY KEY AUTOINCREMENT,"
15        " created_at TEXT NOT NULL,"
16        " layout_id INTEGER NOT NULL,"
17        " grade_id INTEGER NOT NULL,"
18        " source TEXT NOT NULL,"
```

```

18     " rating INTEGER NOT NULL,"
19     " hole_count INTEGER NOT NULL,"
20     " tokens TEXT NOT NULL,"
21     " hole_ids TEXT NOT NULL"
22     ");";
23 // ...
24 }

```

Listing 1: Inicjalizacja tabeli saved_routes w lokalnej bazie wyników

```

1 bool load_holes_from_db(const std::string& db_path, int
    product_size_id,
2                          std::map<int, Hole>& out_holes, std::string&
                          error) {
3     sqlite3* db = nullptr;
4     if (sqlite3_open(db_path.c_str(), &db) != SQLITE_OK) {
5         error = "Could not open DB: " + std::string(sqlite3_errmsg(db));
6         if (db) sqlite3_close(db);
7         return false;
8     }
9
10    const char* sql =
11        "SELECT DISTINCT h.id, h.x, h.y "
12        "FROM product_sizes_layouts_sets psls "
13        "JOIN placements p ON p.layout_id = psls.layout_id AND p.set_id
        = psls.set_id "
14        "JOIN holes h ON h.id = p.hole_id "
15        "WHERE psls.product_size_id = ?";
16    // sqlite3_prepare_v2, bind, step ...
17 }

```

Listing 2: Odczyt pozycji dziurek z bazy boardlib (JOIN trzech tabel)

Struktura aplikacji

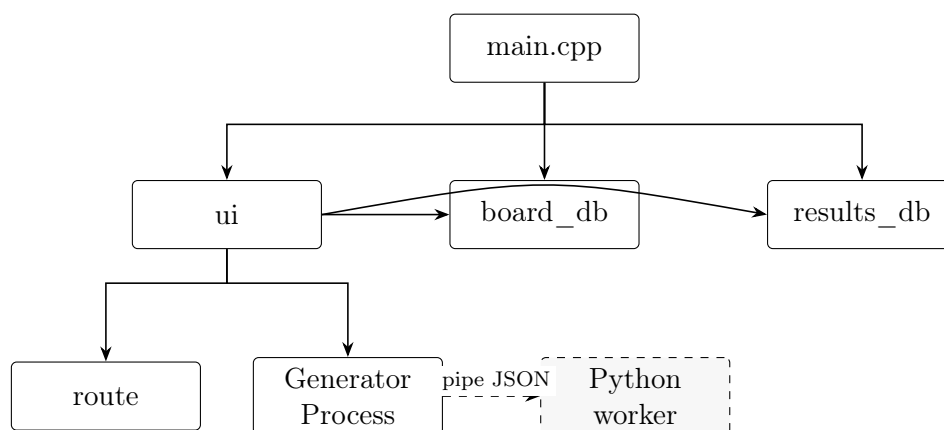
Organizacja modułów

Po reorganizacji projektu kod podzielono na moduły z wyraźnym podziałem `include/` / `src/`:

Moduł	Odpowiedzialność
<code>types.hpp</code>	Struktury <code>Hole</code> , <code>SavedRoute</code> , <code>AppState</code>
<code>board_db</code>	Odczyt dziurek z <code>kilter.sqlite3</code>
<code>results_db</code>	Zapis i odczyt historii tras użytkownika
<code>generator_process</code>	Klasa <code>GeneratorProcess</code> — IPC z workerem Pythona
<code>route</code>	Parsowanie tokenów tras i przypisywanie ról chwyków
<code>ui</code>	Interfejs GTK4, rysowanie planszy (Cairo), obsługa zdarzeń
<code>main.cpp</code>	Inicjalizacja ścieżek, uruchomienie <code>GtkApplication</code>

Program jest napisany obiektowo w sensie modularnym: logika enkapsulowana w klasach (np. `GeneratorProcess`) oraz w funkcjach powiązanych z konkretnym obszarem danych (bazy, trasa, UI). Stan aplikacji zebrany jest w strukturze `AppState`.

Diagram modułów



Rysunek 2: Zależności między modułami aplikacji `kilter_gui`.

Komunikacja z procesem generatora

Wymaganie poziomu II (dwa procesy komunikujące się przez potok) realizuje klasa `GeneratorProcess`. Zamiast uruchamiać interpreter Pythona przy każdym kliknięciu, aplikacja:

1. tworzy parę potoków (`pipe`),
2. wywołuje `fork()` i w procesie potomnym `exec1()` na skrypt `kilter_dl/generate_server.py`,
3. przekierowuje `stdin/stdout` workera na końce potoków,
4. wymienia komunikaty JSON linia po linii (żądanie `generate`, odpowiedź `ok/error`, na koniec `shutdown`).

Listing 3 pokazuje kluczowe fragmenty startu i wysyłania żądania. Obsługa w GUI znajduje się w `on_generate_clicked` (listing 4).

```

1 bool GeneratorProcess::start(...) {
2     int in_pipe[2] = {-1, -1};
3     int out_pipe[2] = {-1, -1};
4     if (pipe(in_pipe) != 0 || pipe(out_pipe) != 0) {
5         last_error_ = "Failed to create pipes for generator process.";
6         return false;
7     }
8
9     pid_t pid = fork();
10    if (pid == 0) {
11        dup2(in_pipe[0], STDIN_FILENO);
12        dup2(out_pipe[1], STDOUT_FILENO);
13        execl(python_path.c_str(), python_path.c_str(),
14              script_path.c_str(),
15              "--token-map", token_map_path.c_str(), "--id-map",
16              id_map_path.c_str(),
17              static_cast<char*>(nullptr));
18        _exit(127);
19    }
20    // handshake: oczekiwanie na {"status":"ready"}
21
22 bool GeneratorProcess::request(const std::string& json_line,
23                                std::string& response_out) {
24     fputs(json_line.c_str(), writer_);
25     fputc('\n', writer_);
26     fflush(writer_);
27     return read_line(response_out);
28 }

```

Listing 3: Uruchomienie workera Pythona i komunikacja przez potoki (IPC)

```

1 if (!state->generator.running()) {
2     state->generator.start(state->python_path,
3                           state->generate_server_script_path,
4                           state->token_map_path,
5                           state->id_map_path,
6                           state->repo_root);
7 }
8
9 std::stringstream req;
10 req << "{\"cmd\":\"generate\","
11      << "\"checkpoint\":\"" << state->checkpoint_path << "\",\"
12      << "\"layout_id\":\"" << layout_id << "\",\"
13      << "\"grade_id\":\"" << grade_id << "\",\"
14      << "\"greedy\":\"" << (greedy ? "true" : "false") << "\"}";
15
16 std::string response;
17 state->generator.request(req.str(), response);
18 // parsowanie JSON: tokens / hole_ids -> rysowanie trasy

```

Listing 4: Wysłanie żądania generacji trasy z poziomu GUI

Działanie aplikacji

Budowanie i uruchomienie

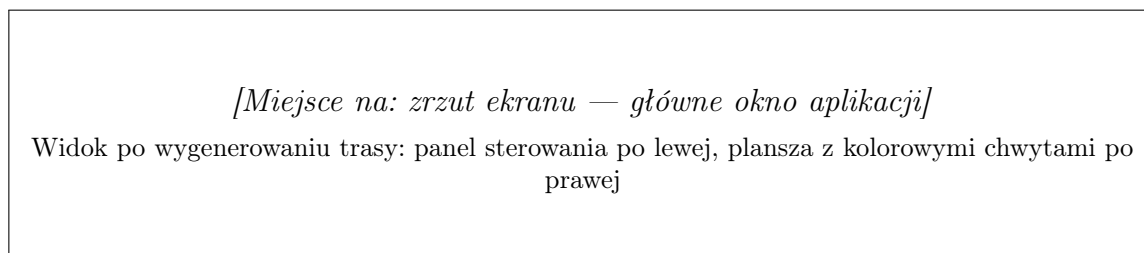
```
./kilter_cpp_gui/run.sh          # build + run z katalogu głównego repo
./kilter_cpp_gui/run.sh build    # tylko kompilacja
```

Aplikacja wymaga: `boardlib_data/kilter.sqlite3`, środowiska Python `kilter_dl/.venv`, co najmniej jednego checkpointu w `kilter_dl/checkpoints/` oraz plików map tokenów w `dataset/`.

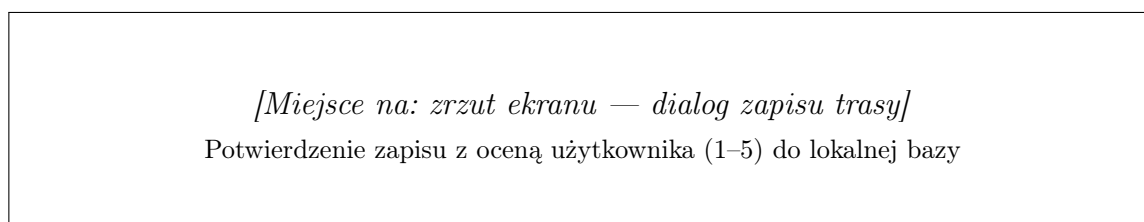
Przebieg użytkownika

1. Użytkownik wybiera układ ściany (**Board**) i trudność (**Grade**).
2. Przycisk *Generate Route + Render* uruchamia (lub ponownie wykorzystuje) proces generatora i wysyła żądanie JSON z parametrami dekodowania.
3. Odpowiedź zawiera tokeny trasy; moduł `route` przypisuje role chwytom (start — zielony, pośredni — niebieski, finish — fioletowy, noga — pomarańczowy).
4. Plansza po prawej stronie rysuje wszystkie dziurki oraz aktywną trasę (Cairo).
5. Użytkownik ustawia ocenę 1–5 i klika *Save route + rating to DB*; po potwierdzeniu w dialogu rekord trafia do `saved_routes.db`.
6. Panel *Recently saved routes* odświeża listę ostatnich 10 zapisów.
7. Alternatywnie: *Load Random Route from Dataset* pobiera trasę ze zbioru bez uruchamiania modelu.

Zrzuty ekranu



Rysunek 3: Główne okno aplikacji GENCLIMB po wygenerowaniu trasy.



Rysunek 4: Dialog potwierdzenia zapisu trasy do bazy wyników.

[Miejsce na: zrzut ekranu — panel historii zapisów]

Lista ostatnich tras z `saved_routes.db` (layout, grade, źródło, liczba chwytów, ocena)

Rysunek 5: Panel historii zapisanych tras.

Podsumowanie

Zrealizowano aplikację C++ z graficznym interfejsem GTK4 do pracy z trasami wspinaczkowymi na ścianach Kilter. Projekt łączy odczyt danych geometrycznych z bazy boardlib, generację tras przez długożyjący proces potomny (IPC przez potoki) oraz trwałe zapis ocen użytkownika w osobnej bazie SQLite.

Najciekawszy element — architektura z procesem workera Pythona: jednokrotne załadowanie modelu PyTorch skraca czas kolejnych generacji w porównaniu z uruchamianiem nowego interpretera przy każdym żądaniu. Protokół JSON linia-po-linii upraszcza debugowanie kanału komunikacji.

Najtrudniejszy element — spójne mapowanie tokenów modelu na identyfikatory dziurek i role chwytów oraz synchronizacja wielu ścieżek danych (model, dataset, baza planszy) przy zachowaniu czytelnej struktury modułowej po refaktoryzacji katalogów `include/` i `src/`.

Obszar do rozbudowy — zgodnie z kryteriami poziomu I warto dodać moduł statystyk (np. średnia/mediana ocen i liczby chwytów w `saved_routes`) oraz interaktywne sortowanie widocznej historii zapisów.